

P1L1 - Benchmark 3D OpenSeesPy

Sector P1L1-S01: pano idealizado entre ejes F-G y 2-3
Semana 1 - Laboratorio de Analisis Estructural 3D

Grupo: MCOC-grupo1
Curso: Analisis Estructural
Fecha de entrega: Jueves 27 de agosto 2026

Documento generado con ayuda de OpenCode (IA)

1. QUE MODELAMOS

Objetivo

Construir y verificar un caso estructural 3D propuesto por el grupo. Se trata de un marco tridimensional de al menos un vano en cada direccion, con losa cuya carga se descarga sobre las vigas por areas tributarias.

Sector escogido: P1L1-S01

Se modela el pano idealizado entre ejes F-G y 2-3 del edificio. Es una zona repetitiva y facil de verificar antes de pasar al edificio completo.

Geometria

Lx = 6.0 m (eje F -> G)
 Ly = 4.0 m (eje 2 -> 3)
 H = 3.0 m (altura del nivel)
 $A_{losa} = 6.0 * 4.0 = 24.0 \text{ m}^2$

Nodos

1 = F2 base = (0, 0, 0)	5 = F2 sup = (0, 0, 3)
2 = G2 base = (6, 0, 0)	6 = G2 sup = (6, 0, 3)
3 = G3 base = (6, 4, 0)	7 = G3 sup = (6, 4, 3)
4 = F3 base = (0, 4, 0)	8 = F3 sup = (0, 4, 3)

Secciones

Columnas P. 70x70: $A = 0.49 \text{ m}^2$, $I_y = I_z = 0.00496 \text{ m}^4$
 Vigas V. 60/80: $A = 0.48 \text{ m}^2$, $I_y = 0.0256 \text{ m}^4$, $I_z = 0.0144 \text{ m}^4$

Material

$E = 25 \text{ GPa}$ (concreto)
 $\nu = 0.20$
 $G = E / (2 * (1 + \nu)) = 10.42 \text{ GPa}$

Apoyos

Los 4 nodos de base (1,2,3,4) estan empotrados:

fix(node, 1, 1, 1, 1, 1) -> restringe los 6 GDL

Esto significa que no hay desplazamiento ni rotacion en base.

Carga de losa

La losa NO se modela con elementos finitos. Se transfiere su carga a las vigas mediante areas tributarias:

$q_G = 7.35 \text{ kN/m}^2$ (250+200+300 kgf/m² aprox.)
 $P_{total} = 7.35 * 24.0 = 176.4 \text{ kN}$
 $A_{trib_por_viga} = 24.0 / 4 = 6.0 \text{ m}^2$
 Vigas de 6m: $w = 7.35 * 6 / 6 = 7.35 \text{ kN/m}$
 Vigas de 4m: $w = 7.35 * 6 / 4 = 11.025 \text{ kN/m}$
 Verificacion: $2 * (7.35 * 6) + 2 * (11.025 * 4) = 176.4 \text{ kN}$

2. IMPORTACIONES Y CONSTANTES

Librerias

```
from __future__ import annotations # Tipos modernos
import csv                         # Exportar CSV
import json                        # Exportar JSON
import math                        # hypot, sqrt
from pathlib import Path           # Rutas de archivos
import matplotlib                  # Graficos
matplotlib.use("Agg")              # Sin pantalla
import matplotlib.pyplot as plt    # Interfaz de graficos
import openseespy.opensees as ops  # Motor de analisis
from mpl_toolkits.mplot3d import Axes3D # Proyeccion 3D
```

Constantes de conversion

```
KN = 1_000.0      # 1 kN = 1000 N
MPa = 1.0e6       # 1 MPa = 1e6 Pa
```

Funciones auxiliares

```
def kN(value_newton):
    return value_newton / KN

def kN_m(value_newton_meter):
    return value_newton_meter / KN
```

Razon: OpenSees trabaja en SI (m, N, Pa). Las funciones kN() y kN_m() convierten los resultados para imprimir valores legibles.

3. FUNCION: rectangular_section()

Que hace

Calcula propiedades elasticas de una seccion rectangular: area (A), inercia respecto a eje Y (Iy), inercia respecto a eje Z (Iz), y constante de torsion (J).

Codigo

```
def rectangular_section(width, height):
    area = width * height
    iy = width * height**3 / 12.0
    iz = height * width**3 / 12.0
    torsion_j = iy + iz
    return {'A': area, 'Iy': iy, 'Iz': iz, 'J': torsion_j}
```

Que significan Iy e Iz

Iy = inercia para flexion alrededor del eje local Y.

Iz = inercia para flexion alrededor del eje local Z.

Para una viga de 0.60 x 0.80 m:

$$Iy = 0.60 * 0.80^3 / 12 = 0.0256 \text{ m}^4 \text{ (flexion fuerte)}$$

$$Iz = 0.80 * 0.60^3 / 12 = 0.0144 \text{ m}^4 \text{ (flexion debil)}$$

Para una columna de 0.70 x 0.70 m:

$$Iy = Iz = 0.70^4 / 12 = 0.0196 \text{ m}^4 \text{ (simetrica)}$$

Resultados

```
column = rectangular_section(0.70, 0.70)
# A=0.49, Iy=0.0196, Iz=0.0196, J=0.0392
```

```
beam = rectangular_section(0.60, 0.80)
# A=0.48, Iy=0.0256, Iz=0.0144, J=0.04
```

4. FUNCIONES VECTORIALES 3D

unit() - Normalizar vector

```
def unit(vector):
    norm = sqrt(sum(v*v for v in vector))
    return tuple(v / norm for v in vector)
```

Convierte cualquier vector a longitud 1. Necesario para ejes locales.

cross() - Producto cruz

```
def cross(a, b):
    return (a[1]*b[2] - a[2]*b[1],
            a[2]*b[0] - a[0]*b[2],
            a[0]*b[1] - a[1]*b[0])
```

Producto cruz: calcula un vector perpendicular a dos vectores dados.

local_axes() - Ejes locales del elemento

```
def local_axes(start, end, vecxz):
    local_x = unit(end - start)      # Eje X = elem_i -> elem_j
    local_y = unit(cross(vecxz, x))  # Eje Y perpendicular a X y vecxz
    local_z = unit(cross(x, y))      # Eje Z completa la terna
    return local_x, local_y, local_z
```

Que es geomTransf y vecxz

geomTransf define la transformacion geometrica del elemento:

como se mapean las coordenadas locales a globales.

vecxz es un vector auxiliar que define la orientacion. Dice 'hacia donde apunta el eje Z local' (o un vector en el plano XZ local).

Columnas: vecxz = (1, 0, 0) -> eje Z local apunta en X global

Vigas: vecxz = (0, 0, 1) -> eje Z local apunta en Z global

5. FUNCION: add_beam_gravity_load()

Que hace

Aplica una carga distribuida vertical sobre un elemento viga 3D. Usa eleLoad con tipo -beamUniform en coordenadas locales.

Codigo

```
def add_beam_gravity_load(ele_tag, line_load):
    ops.eleLoad('-ele', ele_tag, '-type',
               '-beamUniform', 0.0, -line_load, 0.0)
    return 0.0, 0.0, -line_load
```

Argumentos de -beamUniform

eleLoad -beamUniform recibe 3 componentes en LOCALES:

- wy = carga perpendicular en eje Y local (corte)
- wz = carga perpendicular en eje Z local (vertical para vigas)
- wx = carga axial

Para carga gravitacional vertical en vigas horizontales:

wy = 0, wz = -line_load (negativo = hacia abajo), wx = 0

Carga tributaria

```
slab_area = lx * ly = 6.0 * 4.0 = 24.0 m2
q_g = 7.35 kN/m2
tributary_area_per_beam = 24.0 / 4 = 6.0 m2
```

Vigas en X (6m): $w = 7.35 * 6 / 6 = 7.35 \text{ kN/m}$

Vigas en Y (4m): $w = 7.35 * 6 / 4 = 11.025 \text{ kN/m}$

6. NODOS, APOYOS Y MODELO

Crear modelo 3D

```
ops.wipe()
ops.model("basic", "-ndm", 3, "-ndf", 6)
```

ndm = 3: modelo tridimensional.

ndf = 6: cada nodo tiene 6 grados de libertad.

Los 6 GDL son: ux, uy, uz, rx, ry, rz

Los 6 GDL (grados de libertad)

```
DOF 1 = ux  (traslacion horizontal X)
DOF 2 = uy  (traslacion horizontal Y)
DOF 3 = uz  (traslacion vertical Z)
DOF 4 = rx  (rotacion alrededor de X)
DOF 5 = ry  (rotacion alrededor de Y)
DOF 6 = rz  (rotacion alrededor de Z)
```

Crear nodos

```
for node, coords in nodes.items():
    ops.node(node, *coords)
```

Fijar apoyos

```
for node in (1, 2, 3, 4):
    ops.fix(node, 1, 1, 1, 1, 1, 1)
```

fix(nodo, 1,1,1,1,1,1) = empotrado: los 6 GDL estan restringidos.

No hay ni traslacion ni rotacion en la base.

Los nodos superiores (5,6,7,8) estan libres.

7. TRANSFORMACIONES GEOMETRICAS

Definicion

```
ops.geomTransf("Linear", 1, 1.0, 0.0, 0.0) # Columnas
ops.geomTransf("Linear", 2, 0.0, 0.0, 1.0) # Vigas
```

Que es geomTransf

geomTransf define la transformacion geometrica de un elemento:
como se relacionan los ejes locales del elemento con el sistema global.

Linear = sin actualizar geometria (small strains).
Es suficiente para analisis lineal elastico.

Que es el vector vecxz

El tercer argumento es un vector auxiliar que define la orientacion.
Dice 'hacia donde apunta el eje Z local' o un vector en el plano XZ.

Columnas (tag 1): vecxz = (1, 0, 0)
-> eje Z local apunta en direccion X global
-> eje Y local queda perpendicular

Vigas (tag 2): vecxz = (0, 0, 1)
-> eje Z local apunta en direccion Z global (vertical)
-> eje Y local queda perpendicular

Diferencia local vs global

Global: coordenadas del modelo completo (X, Y, Z).
Local: coordenadas de cada elemento (x, y, z).

El eje local x siempre va del nodo i al nodo j.
Los ejes local y y local z dependen de geomTransf.

Las fuerzas localForce se reportan en ejes LOCALES.
Para convertir a globales, se usa la transformacion inversa.

8. ELEMENTOS elasticBeamColumn

Sintaxis 3D

```
ops.element('elasticBeamColumn', tag, ni, nj,
            A, E, G, J, Iy, Iz, transfTag)
```

Parametros

tag = identificador unico del elemento

ni, nj = nodos inicial y final

A = area de la seccion

E = modulo elastico

G = modulo de corte

J = constante de torsion

Iy = inercia para flexion sobre eje Y local

Iz = inercia para flexion sobre eje Z local

transfTag = tag de la geomTransf a usar

Por que Iy e Iz son diferentes en vigas?

Una viga rectangular 60x80 es mas alta que ancha.

$I_y = b \cdot h^3 / 12 = 0.60 \cdot 0.80^3 / 12 = 0.0256 \text{ m}^4$ (flexion fuerte)

$I_z = h \cdot b^3 / 12 = 0.80 \cdot 0.60^3 / 12 = 0.0144 \text{ m}^4$ (flexion debil)

$I_y > I_z$ porque es mas dificil doblar la viga por su lado alto.

Elementos del modelo

Columnas (1-4): elasticBeamColumn ... 1 # transf 1

Vigas (5-8): elasticBeamColumn ... 2 # transf 2

9. CARGAS Y ANALISIS

Patron de carga

```
ops.timeSeries("Linear", 1) # Carga se aplica en 1 paso
ops.pattern("Plain", 1, 1) # Patron usando la serie 1
```

Aplicar cargas en vigas

```
uniform_loads[5] = add_beam_gravity_load(5, line_load_x) # viga eje 2
uniform_loads[7] = add_beam_gravity_load(7, line_load_x) # viga eje 3
uniform_loads[6] = add_beam_gravity_load(6, line_load_y) # viga eje 6
uniform_loads[8] = add_beam_gravity_load(8, line_load_y) # viga eje F
```

Configurar analisis

```
ops.system("BandGeneral") # Matriz de rigidez banda
ops.numberer("RCM") # Numeracion RCM
ops.constraints("Transformation") # Manejo de restricciones
ops.integrator("LoadControl", 1.0) # 1 paso
ops.algorithm("Linear") # Sin iteracion
ops.analysis("Static") # Estatico
```

Ejecutar

```
result = ops.analyze(1)
if result != 0:
    raise RuntimeError('OpenSees no convergio')
```

*analyze(1) resuelve $K*u = F$ para 1 paso de carga.*

Si converge, result = 0. Si falla, result != 0.

Que esta resolviendo OpenSees?

OpenSees ensambla la matriz de rigidez global K a partir de las rigideces de cada elemento (basado en A , E , G , J , I_y , I_z).

Luego resuelve: $K * u = F$

K = matriz de rigidez global

u = vector de desplazamientos nodales (incognita)

F = vector de cargas

Una vez que tiene u , calcula las fuerzas internas de cada elemento.

10. LEER RESULTADOS

Reacciones

```
ops.reactions()

reactions = {
    node: tuple(ops.nodeReaction(node, dof) for dof in (1,2,3))
    for node in (1, 2, 3, 4)
}
```

Desplazamientos

```
displacements = {
    node: tuple(ops.nodeDisp(node, dof) for dof in (1,2,3))
    for node in nodes
}
```

Fuerzas locales (3D)

```
local_forces = {ele: ops.eleResponse(ele, 'localForce')
                for ele in elements}
```

En 3D, *localForce* retorna 12 valores por elemento:

[Ni, Vyi, Vzi, Ti, Myi, Mzi, Nj, Vyj, Vzj, Tj, Myj, Mzj]

N = axial, V = corte, M = momento, T = torsion

Subscript i = nodo inicial, j = nodo final

Desplazamiento maximo superior

```
max_top_uz = max(abs(displacements[node][2])
                 for node in (5,6,7,8))

# Resultado: -1.080e-05 m (compresion de columna)
```

11. DIAGRAMAS DE ESFUERZOS INTERNOS

element_diagram_values()

Para cada elemento, calcula N, V y M en 81 estaciones a lo largo de su longitud usando equilibrio local.

Ecuaciones de equilibrio

Axial:

$$N(x) = N_i + w_x * x$$

Cortante:

$$V_y(x) = V_{yi} + w_y * x$$

$$V_z(x) = V_{zi} + w_z * x$$

$$V_{res} = \sqrt{V_y^2 + V_z^2}$$

Momento (integrando cortes):

$$M_y(x) = M_{yi} + V_{zi} * x + 0.5 * w_z * x^2$$

$$M_z(x) = M_{zi} - V_{yi} * x - 0.5 * w_y * x^2$$

$$M_{res} = \sqrt{M_y^2 + M_z^2}$$

Por que no interpolar linealmente?

En vigas con carga distribuida, el momento es parabolico, no lineal. Si solo interpolamos entre extremos, perderiamos el valor maximo interior. Por eso se calcula por estaciones usando las ecuaciones de equilibrio.

Cierre de diagramas

El script verifica que los diagramas cierren correctamente: el valor en el nodo j debe coincidir con la fuerza de extremo j reportada por OpenSees. Si hay error grande, el modelo tiene un problema.

12. VERIFICACION

Equilibrio vertical

```
total_reaction_z = sum(reactions[n][2] for n in (1,2,3,4))
vertical_residual = total_reaction_z - total_slab_load

# Debe ser ~0:
assert isclose(vertical_residual, 0, abs_tol=1e-6)
```

Reacciones por columna

```
expected = total_slab_load / 4 = 176.4 / 4 = 44.1 kN
# Cada columna debe tomar ~44.1 kN:
assert isclose(reactions[n][2], 44.1, abs_tol=1e-5)
```

Tabla de verificacion

Magnitud	Referencia	OpenSees

Suma cargas verticales	-176.400 kN	-176.400 kN
Suma reacciones verticales	176.400 kN	176.400 kN
Reaccion por columna	44.100 kN	44.100 kN
Axial columna elem 1	44.100 kN	44.100 kN
Max N global diagrama	44.100 kN	44.100 kN
Max Vres global diagrama	22.050 kN	22.050 kN
Max Mres global diagrama	19.250 kN*m	19.250 kN*m
Despl. vertical superior	-1.080e-05 m	-1.080e-05 m
Cierre diagramas fuerza	0 kN	0.000e+00 kN
Cierre diagramas momento	0 kN*m	1.091e-14 kN*m

Por que converger no significa estar correcto?

OpenSees puede converger (result=0) incluso con errores en:

- Signo de cargas (carga hacia arriba en vez de abajo)
- Unidades incorrectas (mm en vez de m)
- Apoyos mal definidos (liberados sin querer)
- Secciones incorrectas

Por eso SIEMPRE hay que comparar contra valores de referencia manuales o de otro programa. El equilibrio es necesario pero no suficiente.

13. PREGUNTAS PARA LA DEFENSA

Los 6 grados de libertad representan los movimientos posibles:

- 3 traslaciones: u_x (X), u_y (Y), u_z (Z)
- 3 rotaciones: r_x (alrededor de X), r_y (alrededor de Y), r_z (alrededor de Z)

En 2D solo hay 3 (u_x , u_y , r_z). En 3D hay 6.

2. Que representa geomTransf?

Define como se calculan las deformaciones del elemento.

Linear = pequenas deformaciones, sin actualizar geometria.

Incluye un vector auxiliar ($vecxz$) que fija la orientacion de los ejes locales Y y Z del elemento.

3. Diferencia local vs global?

Global: sistema de coordenadas del modelo completo (X, Y, Z).

Local: sistema de coordenadas de cada elemento (x, y, z).

El eje local x siempre va del nodo i al nodo j.

Las fuerzas localForce se reportan en ejes locales.

4. Que representa Iy e Iz?

I_y = inercia para flexion alrededor del eje local Y.

I_z = inercia para flexion alrededor del eje local Z.

Para una viga 60x80: $I_y > I_z$ porque es mas dificil doblar por su lado alto.

5. Que esta resolviendo OpenSees?

Resuelve $K \cdot u = F$:

- K: matriz de rigidez global (ensamblada de cada elemento)
- u: vector de desplazamientos nodales (incognita)
- F: vector de cargas

Con u conocido, calcula fuerzas internas de cada elemento.

6. Por que converger no es suficiente?

OpenSees puede converger con errores de:

- Signos de carga invertidos
- Unidades inconsistentes
- Apoyos mal definidos
- Secciones incorrectas

Siempre hay que verificar equilibrio, unidades y comparar contra valores de referencia.

14. COMANDOS UTILES

Ejecutar el script

```
cd entregas/p1l1_benchmark_3d
python opensees/benchmark_3d.py
```

Ejecutar desde VS Code

Abrir terminal con Ctrl+` y pegar el comando anterior

Instalar dependencias

```
python -m pip install openseespy
python -m pip install matplotlib
python -m pip install fpdf2
```

Git

```
git pull          # Bajar cambios del grupo
git add .         # Seleccionar todos los cambios
git commit -m "msg" # Guardar cambios localmente
git push         # Subir a GitHub
```

Ubicacion de archivos

```
entregas/p1l1_benchmark_3d/
  opensees/benchmark_3d.py          # El script
  results/geometria_deformada_ejes.png # Geometria 3D
  results/diagramas_nvm_3d.png      # Diagramas NVM
  results/fuerzas_elementos.csv     # Fuerzas locales
  results/diagramas_nvm_3d_valores.csv # Valores por estacion
  results/verificacion.json         # Chequeos
  docs/semana01.md                 # Informe
  docs/P1L1_explicacion_codigo.pdf  # Este PDF
```